1. Problem Title & Source

Title: **Odd Even Vehicle Fine Calculation**

Source: **TCS NQT Previous Coding Questions**

2. Problem Statement (Clear Version)

To control pollution in Delhi, the government implemented an **Odd–Even vehicle rule**.

The rule works as follows:

- Vehicles with **even last digit** in their registration number can travel on **even dates**.
- Vehicles with **odd last digit** can travel on **odd dates**.

If a vehicle violates this rule, the traffic police impose a **fine**.

You are given:

- N → number of vehicles on the road
- a[i] → last digit of each vehicle's registration number
- D → the date
- X → fine amount charged per violating vehicle

Your task is to **calculate the total fine collected from vehicles violating the Odd–Even rule**.

3. Examples

Example 1

Input

4

5 2 3 7

12

200

Output

600

Explanation

Date = **12 (even)**

Allowed vehicles:

even registration digits

Violating vehicles:

5, 3, 7

Number of violations:

3

Total fine:

3 × 200 = 600

Example 2

Input

5

2 4 6 1 3

9

100

Output

300

Explanation

Date = **9 (odd)**

Allowed vehicles:

odd numbers

Violating vehicles:

2, 4, 6

Violations = **3**

Fine:

3 × 100 = 300

4. Constraints

$0 < N \leq 100$

$1 \leq a[i] \leq 9$

$1 \leq D \leq 30$

$100 \leq X \leq 5000$

Observations

- Only the **parity (odd/even)** of numbers matters
- Straightforward counting problem

5. Core Concept (Pattern)

Array Traversal + Counting

Key idea:

Check parity of each vehicle digit

6. Thought Process

Step 1 — Determine Allowed Vehicles

If

D % 2 == 0

Even vehicles allowed.

If

D % 2 == 1

Odd vehicles allowed.

Step 2 — Identify Violations

A violation occurs when:

vehicle parity ≠ date parity

Step 3 — Calculate Fine

Total fine:

violations × X

7. Visualization

Example

vehicles = [5,2,3,7]

date = 12

Allowed

even numbers

Check vehicles

Vehicle	Allowed?	Violation
5	No	Yes
2	Yes	No
3	No	Yes
7	No	Yes

Violations

3

Total fine

3 × 200 = 600

8. Pseudocode

read N

read array

read D

read X

```
violations = 0

for each vehicle

   if D is even AND vehicle is odd
      violations++

   if D is odd AND vehicle is even
      violations++

totalFine = violations × X

print totalFine
```

9. Code Implementation

Python

```
n = int(input())
arr = list(map(int,input().split()))

d = int(input())
x = int(input())

violations = 0

for v in arr:

   if d % 2 == 0 and v % 2 == 1:
      violations += 1

   elif d % 2 == 1 and v % 2 == 0:
      violations += 1

print(violations * x)
```

Java

```
import java.util.*;
```

```
class Main {

    public static void main(String[] args){

        Scanner sc = new Scanner(System.in);

        int n = sc.nextInt();

        int[] arr = new int[n];

        for(int i=0;i<n;i++)
            arr[i] = sc.nextInt();

        int d = sc.nextInt();
        int x = sc.nextInt();

        int violations = 0;

        for(int v : arr){

            if(d % 2 == 0 && v % 2 == 1)
                violations++;

            else if(d % 2 == 1 && v % 2 == 0)
                violations++;
        }

        System.out.println(violations * x);
    }
}
```

10. Time & Space Complexity

Time Complexity

O(N)

Single traversal of the array.

Space Complexity

O(1)

Only counters used.

11. Common Mistakes

1. Confusing **allowed vs violating vehicles**
2. Using incorrect parity condition
3. Forgetting to multiply by fine amount
4. Handling input format incorrectly

12. Detailed Dry Run

Input

N = 4

arr = [5,2,3,7]

D = 12

X = 200

Check

Vehicle	Parity	Violation
5	Odd	Yes
2	Even	No
3	Odd	Yes
7	Odd	Yes

Violations

3

Total fine

600

13. Real-Life Applications

Parity-based rules appear in:

- Traffic regulation
- Load balancing
- Alternating scheduling
- Resource allocation systems

14. Common Traps

1. Checking wrong parity condition
2. Using = instead of ==

3. Forgetting edge cases like N=1
4. Misreading input order

15. Related Problems

Parity-based problems:

LC 905 Sort Array By Parity

LC 922 Sort Array By Parity II

LC 136 Single Number

16. Alternate Approaches

Approach	Complexity	Notes
Direct counting	O(N)	optimal
Filtering arrays	O(N)	unnecessary extra memory

17. Why This Solution Works

The Odd–Even rule depends solely on **parity comparison**.

Checking parity using modulo operation:

number % 2

allows us to determine violations efficiently.

18. Variations / Follow-Up Questions

Possible interview variations:

1. Count vehicles **allowed instead of fined**
2. Calculate **total vehicles allowed**
3. Apply rule to **multiple days**
4. Apply rule with **different fine per vehicle type**